IX SWAP

SMART CONTRACT AUDIT

ZOKYO.

September 6th, 2021 | v. 1.0

PASS

Zokyo's Security Team has concluded that this smart contract passes security qualifications to be listed on digital asset exchanges

TECHNICAL SUMMARY

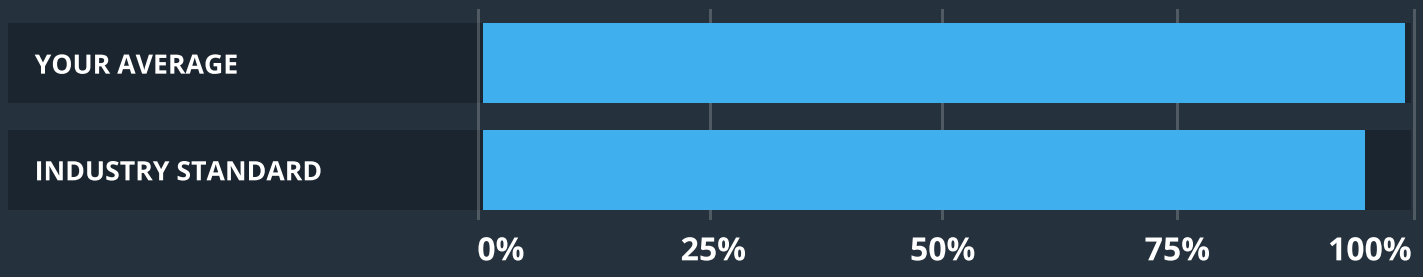
This document outlines the overall security of the IXS smart contracts, evaluated by Zokyo's Blockchain Security team.

The scope of this audit was to analyze and document the IXS smart contract codebase for quality, security, and correctness.

Contract Status

There were no critical issues found during the audit.

Testable Code



The testable code is 99.37%, which is above the industry standard of 95%.

It should be noted that this audit is not an endorsement of the reliability or effectiveness of the contract, rather limited to an assessment of the logic and implementation. In order to ensure a secure contract that's able to withstand the Ethereum network's fast-paced and rapidly changing environment, we at Zokyo recommend that the IXS team put in place a bug bounty program to encourage further and active analysis of the smart contract.

TABLE OF CONTENTS

Auditing Strategy and Techniques Applied 3
Summary 4
Structure and Organization of Document 5
Complete Analysis 6
Code Coverage and Test Results for all files 9
Tests written by Zokyo Secured team 9

AUDITING STRATEGY AND TECHNIQUES APPLIED

The Smart contract's source code was taken from the IXS repository.

Repository (archive hash):
6e339bb351632d60a95cd8cb5a9faa316836e1bd

Last commit (archive hash):
78f30ed98ce6b00a05f596b16862ee382aea2ed5

Throughout the review process, care was taken to ensure that the token contract:

- Implements and adheres to existing Token standards appropriately and effectively;
- Documentation and code comments match logic and behavior;
- Distributes tokens in a manner that matches calculations;
- Follows best practices in efficient use of gas, without unnecessary waste;
- Uses methods safe from reentrance attacks;
- Is not affected by the latest vulnerabilities;
- Whether the code meets best practices in code readability, etc.

Zokyo's Security Team has followed best practices and industry-standard techniques to verify the implementation of IXS smart contracts. To do so, the code is reviewed line-by-line by our smart contract developers, documenting any issues as they are discovered. Part of this work includes writing a unit test suite using the Truffle testing framework. In summary, our strategies consist largely of manual collaboration between multiple team members at each stage of the review:

1	Due diligence in assessing the overall code quality of the codebase.	3	Testing contract logic against common and uncommon attack vectors.
2	Cross-comparison with other, similar smart contracts by industry leaders.	4	Thorough, manual review of the codebase, line-by-line.

SUMMARY

There were no critical issues found during the audit. Among the findings we have two issues (one informational and one with high severity). All of the findings were resolved by the IXS team and bear no operational or security risk for the end user or contract owner.

Contracts are well written and structured. Evaluating the manual report and testing part, we can state that the contracts are safe to be used and fully production ready. Based on this fact Zokyo auditing team has given a score of 99 to the given contracts.

STRUCTURE AND ORGANIZATION OF DOCUMENT

For ease of navigation, sections are arranged from most critical to least critical. Issues are tagged “Resolved” or “Unresolved” depending on whether they have been fixed or addressed. Furthermore, the severity of each issue is written as assessed by the risk of exploitation or other unexpected or otherwise unsafe behavior:

Critical

The issue affects the ability of the contract to compile or operate in a significant way.

High

The issue affects the ability of the contract to compile or operate in a significant way.

Medium

The issue affects the ability of the contract to operate in a way that doesn’t significantly hinder its behavior.

Low

The issue has minimal impact on the contract’s ability to operate.

Informational

The issue has no impact on the contract’s ability to operate.

COMPLETE ANALYSIS

Reentrancy to function which is unallowed access for external senders

HIGH | RESOLVED

- The issue has been resolved by modifying _stakeFor() and _unstake() to be private. Based on the test file IxsReturningStakeBankPostIdoV1.spec the dev showed that functions Stake and StakeFor are undesired to be called from outside by senders despite being public:

```
it('should not allow users to use Stake and StakeFor functions', async () => {
  await expect(bank.connect(treasury).stakeFor(treasury.address, STAKE_AMOUNT, EMPTY_DATA))
  .to.be.revertedWith('IxsStakeBank: FORBIDDEN')
  await expect(bank.connect(treasury).stake( STAKE_AMOUNT, EMPTY_DATA))
  .to.be.revertedWith('IxsStakeBank: FORBIDDEN')
```

- The developer forced that by putting guards around the function which are not allowing it to work except from inside the contract;
- Unfortunately there is a possible way for a malicious ERC20 token to do that undesired action based on the following:
  - A call to stakeForWeek for instance would trigger call stack until we end up here;
  - StakingUnlocked allow stakeFor to be called by unlocking the guard:

```
function stakeForWeek(
    address user,
    uint128 amount,
    bytes calldata data
  ) external lock {
    require(amount > 0, 'IxsStakeBank: LOW_AMOUNT');
    // updating historical pool size for the period
    oneWeekHistoricalPoolSize = uint128(oneWeekHistoricalPoolSize.add(amount));
    // saving Stake transaction in the storage and staking via stakeFor from
```

```
IxsReturningStakeBank.sol
_addStakeTransaction(stakesByPeriod[user][uint8(StakingPeriod.oneWeek)], amount);
    stakingUnlocked = 1;
    stakeFor(user, uint256(amount), data);
    stakingUnlocked = 0;
  }
```

- A malicious token with a malicious implementation for transferFrom inside function _safeClaimTokens can in this situation call stake while the guard allows it:

```
function stakeFor(
    address user,
    uint256 amount,
    bytes memory data
  ) public whenNotPaused {
    _updateCheckpointAtNow(stakesFor[user], amount, false);
    _updateCheckpointAtNow(stakeHistory, amount, false);
    _safeClaimTokens(token, msg.sender, amount);
    emit Staked(user, amount, totalStakedFor(user), data);
  }
```

- stake can be called unguarded:

```
function stake(uint256 amount, bytes memory data) public {
    stakeFor(msg.sender, amount, data);
  }
```

Recommendation:
If that function is essential to be public not internal (i.e. wants trusted contacts to call it). Then IXS needs to audit tokens entering the pool assuring they are not malicious.

Excessive use of duplicated code

INFORMATIONAL | UNRESOLVED

The contract contains duplicated code which is understandable since it actually saves gas when the functions are invoked on-chain. But the test code still contains frequently duplicated snippets. This remains though an Informational and opinionated issue as it does not affect the functionality in anyway.

Duplicated code leaves the codebase prone to the introduction of errors. Errors can inadvertently be introduced when functionality changes are not replicated across all instances of code that should be identical. Logic is duplicated between different staking periods stakeForWeek, stakeForMonth, stakeForTwoMonths, stakeForThreeMonths in test file. Here's an example of a duplicated snippet of test code.

```
it('should not allow user to unstake twice the same stake nor unstake non-stake index (Perl
  await bank.connect(treasury).stakeForWeek(treasury.address, STAKE_AMOUNT, EMPTY_DATA)

  let firstBlock = await provider.getBlockNumber()
  let theBlock = provider.getBlock(firstBlock)
  let blockTime = (await theBlock).timestamp
  let secondBlock = firstBlock + 5

  expect(await bank.totalStakedFor(treasury.address)).to.eq(STAKE_AMOUNT)
  expect(await govToken.balanceOf(treasury.address)).to.eq(STAKE_AMOUNT)

  await mineBlock(provider, blockTime + (3600 * 24 * 7)) // 7 days in future
  await advanceToBlock(provider, secondBlock)

  await expect(bank.connect(treasury).unstakeFromWeek(0, EMPTY_DATA))
    .to.emit(bank, 'Unstaked')
    .withArgs(treasury.address, STAKE_AMOUNT, 0, EMPTY_DATA)

  expect(await bank.totalStakedFor(treasury.address)).to.eq(0)
  expect(await govToken.balanceOf(treasury.address)).to.eq(0)
```

Recommendation:
Write modules as helpers for the test.

CODE COVERAGE AND TEST RESULTS FOR ALL FILES

Tests written by Zokyo Security team

As part of our work assisting IXS in verifying the correctness of their contract code, our team was responsible for writing integration tests using the Truffle testing framework.

Tests were based on the functionality of the code, as well as a review of the IXS contract requirements for details about issuance amounts and how the system handles these.

Code Coverage

The resulting code coverage (i.e., the ratio of tests-to-code) is as follows:

FILE	% STMTS	% BRANCH	% FUNCS	% LINES	UNCOVERED LINES
contracts\	99.37	74.14	100.00	99.37	
IXSReturningStakeBankPostIdoV1.sol	99.37	74.14	100.00	99.37	424
All files	**99.37**	**74.14**	**100.00**	**99.37**	

Test Results

Contract: IxsReturningStakeBankPostIdoV1

- ✓ shouldn't create instance of contract with incorrect values of rewardsVault, ilInsuranceVault and ecosystemWallet (5369ms)
- ✓ shouldn't exceed pool limit while testing (651ms)
- ✓ shouldn't stake zero amount (372ms)
- ✓ shouldn't stake zero amount (363ms)
- ✓ should stake for week (2675ms)
- ✓ should stake for month (2697ms)
- ✓ should stake for two months (2547ms)
- ✓ should stake for three months (2447ms)
- ✓ should unstake from week (9539ms)
- ✓ shouldn't unstake when the staked amount is zero (13159ms)

```
✓ shouldn't  unstake when the lock period is not passed (7881ms)
✓ should unstake from month (9973ms)
✓ should unstake from two months (8637ms)
✓ should  unstake reward from two months before maturity (5609ms)
✓ should unstake from three months (4982ms)
✓ should unstake from three months before maturity (5313ms)
✓ shouldn't call stakeFor() directly (90ms)
✓ shouldn't  call unstake() directly (99ms)
✓ shouldn't call stake() directly (93ms)
✓ should show vesting transaction details of the staker correct (5405ms)
✓ should start vesting after unstake - Period One week (5111ms)
✓ should claim available rewards (5857ms)
✓ should show payouts for the investor for each vesting-transaction correct (5414ms)
✓ should destroy contract by pauser (1819ms)
✓ shouldn't destroy contract with non zero balance of Ixs token (114ms)

25 passing (5m)
```

We are grateful to have been given the opportunity to work with the IXS team.

The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

Zokyo's Security Team recommends that the IXS team put in place a bug bounty program to encourage further analysis of the smart contract by third parties.

ZOKYO.